

Chapter 8. Improving How an Agent Learns

Complex industrial problems can often be decomposed into *directed acyclic graphs* (DAGs). This helps development productivity, by splitting one long project into many smaller projects that are easier to solve. It helps reinforcement learning (RL) because smaller components are often easier and quicker to train and can be more robust. [“Hierarchical Reinforcement Learning”](#) shows one common formalism, which is to derive a hierarchy of policies, where low-level policies are responsible for fine-grained “skills” and high-level policies do long-term planning.

So far in this book I have only considered single-agent problems. Some problems need teams of agents, or at least there may be multiple agents operating within the same environment. [“Multi-Agent Reinforcement Learning”](#) shows you how agents can cooperate or compete to solve multi-agent problems with global or local rewards.

Another rapidly evolving area of RL is redefining how you should think about rewards. Traditionally, it is the sole responsibility of the agent to use the reward signal to learn a policy. But you can augment this process by providing extra, potentially external information in the form of expert guidance. [“Expert Guidance”](#) discusses how to incorporate expertise into policies that ultimately help to speed up learning by improving exploration. It’s even possible to *learn* the optimal reward, which aims to negate the need for reward engineering.

But first, RL is inextricably linked to Markov decision processes (MDPs) by the Bellman equation, and all RL algorithms rely on theoretical guarantees of optimality when making sequential decisions (you might want to review [“Markov Decision Processes”](#)). But a variety of real-life scenarios break the underlying assumptions of an MDP, which often leads to the silent failure of the assumptions made by the Bellman equation and manifest as policies that won’t converge, no matter how much hyperparame-

ter tinkering you do. The next section investigates variations on the basic MDP to accommodate these scenarios.

NOTE

This chapter summarizes the state-of-the-art. I think that the four most important themes are how you define the MDP, hierarchical RL, multi-agent RL, and providing expert guidance. But all of these can be considered as “advanced” and if discussed in full would take another book. However, I do think it is useful to present an overview so that you know it is there and can dig deeper if you want to. As usual, the citations and recommendations in [“Further Reading”](#) can provide a wealth of extra information.

Rethinking the MDP

RL algorithms that leverage the MDP framework and the resulting guarantees from the Bellman equation assume that the underlying environment is Markovian. Although there may be a great deal of uncertainty about the effects of an agent’s actions, the perception of the state of the environment must be perfect. The agent then uses this omnipotent knowledge to choose the next action according to the current policy, or $\pi(a \mid s)$.

But what if the state representation is not perfect? What if when you tell the servo to move to a position, it moves to a position that is randomly distributed about that position? Or what if there are latent, hidden features that represent how likely a person is to click a recommendation? In these situations the state is not fully observable and the Markovian assumptions are broken.

Partially Observable Markov Decision Process

One solution to this *partial observability* is to split the representation of the state. A partially observable Markov decision process (POMDP) includes extra parameters over the MDP to include a finite set of observations that the agent can experience, Ω , and an *observation function*, O , that maps a state and action to a probability distribution over actions. You can then convert the problem back into an MDP by approximating

the inverse of the observation function and using it to convert observations back into states.

[Figure 8-1](#) shows that a POMDP is an extension of the MDP, in which the agent is unable to directly observe the current state of the environment. Instead it uses an observation and information about the previous action and state to generate a prediction of the current state, $\hat{s}$. The prediction of the state is often called the *belief state*, $\mathbf{b}$, which you can describe as “this is the state that I believe the environment is in.”

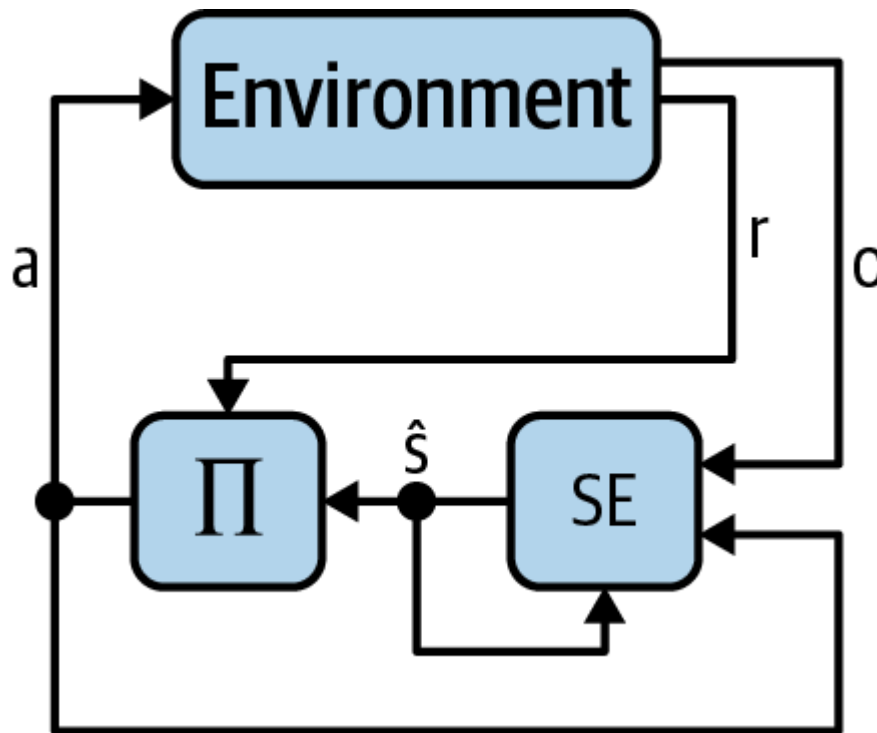


Figure 8-1. You can decompose a POMDP into an MDP (with states, actions, rewards, and a policy) and a state estimator (SE) that predicts states from observations (o).

Predicting the belief state

So what is the belief state and how can you predict it? Consider an example where you were trying to direct a robot through a simple maze. The agent doesn't know the layout of the maze, but it might be intelligent enough to be able to detect that there is a “T-junction” ahead and it needs to make a choice on which direction to take. A naive policy would overfit the training environments and converge to a point where it suggests a single decision every time.

A simple solution to this problem, as indicated by the recurrence in [Figure 8-1](#), is that the best action can be based upon past experience. I don't mean past experience in the training sense, not a replay buffer, but past experience of the current episode and environment. For example, if

you know that the robot has already tried left and that led to a dead end, then the hidden state of the maze (the best route) is probably the option to the right.

Many sophisticated examples leverage this simple idea by including past actions and observations in the state representation. For example, the Atari benchmarks always stack four frames of pixel observations in order to retain some “history” of the environment. This allows the agent to predict trajectories of Pong balls or the movement of enemies. And many robotics examples include previous actions and states to help account for inaccuracies in positioning and smooth out movement.

Although this idea helps in many cases, it is not sufficient in general. In order to act optimally, the agent needs to actively consider how uncertain it is about the current situation. A more sophisticated solution is to model the state estimator using an approximator that is capable of learning time-dependent features.

There are multiple options for solving this problem. You can attempt to track the belief state; a simple version could utilize a Kalman filter although more recent approaches attempt to build neural networks to derive the optimal Bayes filter.¹ You could incorporate an approximation of the belief model directly into the policy model using memory-enabled predictors or belief trackers.^{2,3}

But the simplest (the previous reference had 24 authors!) and most common approach is to slap on a recurrent neural network (RNN) at the end of the state representation function. Depending on the problem, the RNNs don’t need to be complicated and they allow the agent to make reasoned decisions based upon past observations. This allows you to train the RNN using standard techniques and tooling.

As a conceptual example, consider the experiment from [“Industrial Example: Learning to Drive with a Remote Control Car”](#), where I used a variational autoencoder to represent the state. I could add an RNN to the output of the state representation and train it by predicting future observations. Eventually it would be able to provide predictions of what would happen given certain actions and therefore break the partial observability problem. The policy would have full observability of all possible future observations so you can optimize the MDP using standard RL methods.

Case Study: Using POMDPs in Autonomous Vehicles

You might suggest that the previous example is too contrived to apply to real life. But a surprising number of real-life scenarios fit perfectly into the previous model. One such example is that of autonomous vehicles.

A common autonomous task is to decide when it is safe for the vehicle to move; for example, traversing an intersection with stop signs or a roundabout is a common driving situation. Upon arrival, an agent has to time its actions to perfection to avoid hitting other vehicles. Yet if the vehicle waits too long, valuable time is lost and the efficiency of the intersection decreases.

The vehicle has a limited view of the road. The onboard sensors, such as a camera or a LIDAR module, are line of site and cannot see around obstacles. This makes the problem only partially observable, because the agent cannot sense important features outside of this space, like a speeding vehicle hurtling toward the intersection.

Traditional algorithms attempt to quantify the time to collision, but estimating the time accurately and dealing with unexpected situations is hard and not reliable enough for autonomous operation.

By modeling the problem as a POMDP, you can learn an optimal policy using RL. Qiao et al. demonstrate (in a simulation) that an agent with an LSTM-enabled policy is capable of learning when to set off from a four-way intersection to turn left, right, or move straight ahead.⁴ The LSTM policy is able to learn which partially observed situations lead to a collision, via an appropriately engineered reward, and avoid them. You can [see the results of this experiment on YouTube](#).

Qiao et al. went on to add hierarchical RL to the previous model to better model the high-level actions (or low-level subtasks, depending on how you think of it) of go, stop, follow vehicle in front, and so on.⁵ You read more about hierarchical RL in [“Hierarchical Reinforcement Learning”](#).

Contextual Markov Decision Processes

There are many problems in which the underlying MDP is the same, but the contextual situation is different. For example, imagine you were training an agent to administer a drug to a diabetes patient based upon their sugar levels. The underlying MDP is the same for every patient, sugar levels are the observation, and administering the drug is the action. But the observation is affected by external variables, like the gender or age of the patient.⁶

This sounds like a situation similar to POMDPs, but there is a crucial difference. These contextual variables do not change with each measurement. By modeling the context as a set of fixed parameters you constrain the space the agent has to search, which can lead to more robust models and quicker learning.⁷

This extension is crucial when presented with a problem that has a small number of steps in an episode (short Markov chains) but massive numbers of independent episodes. This is precisely the situation that recommendation algorithms find themselves in, which is why most of the contextual MDP (CMDP) research is driven by recommendation or ranking applications.⁸

MDPs with Changing Actions

Some applications have changing action sets and the research has followed two paths that lead to similar results. The first set of research concentrates on situations when actions are not always available. For example, in path planning applications like vehicle or packet routing, paths may be closed due to damage or maintenance, or in advertising, ad selection may change depending on the time of day.⁹

The premise is a stochastic action set (SAS), which includes a set of actions available at a certain time, modeled as a random variable. A common solution to this problem is to learn a policy derived from only the actions observed in that particular state. Boutilier et al. showed that you can then utilize standard Q-learning variants.¹⁰ Chandak et al. extended the idea to incorporate policy gradient methods.¹¹

A second, but more general strand of research focuses more on lifelong learning, or continuous learning. Here, action sets change because problems and priorities change. For example, you might want to add a new capability to your robotic arm or introduce a new set of ads without having to retrain your model and lose that useful experience. The major problem here is the curse of dimensionality, with ever-growing potential action sets and a lack of data to sample those actions. In theory, adding more actions is as simple as adding them to the MDP. But creating an architecture that can handle such a change is challenging.

Chandak et al. demonstrated that this was possible by splitting the policy in two. The first is responsible for making decisions based upon the state, but the output of this is in a meta-space, not the action space. The second component then maps this meta-space to the action space. Together they form a complete policy.¹²

The benefit is that you can alter the meta-action model parameters to account for new actions without touching the state-representation part. Then you can use standard supervised learning on a replay buffer to train the new meta-action model. This limits the amount of retraining required from the live environment to only the parts of the model that impact the new action.

Regularized MDPs

It is worth mentioning again that the class of algorithms that include regularizers are also a reformulation of the MDP. If you recall (see [“Summary”](#)), SAC and other similar algorithms attempt to find an optimal policy that maximizes the discounted return, like any algorithm based upon an MDP, but they also include a regularization term. The idea here is that you can use this term to enforce robustness, similar to how regularizers impact linear regression. But the addition fundamentally alters the Bellman equality, which means there is an impact on the underlying MDP. Thankfully, Li et al. proved that it doesn’t matter what regularization you apply, the algorithm will still converge, so long as you follow some rules defining the regularizing function.¹³

Hierarchical Reinforcement Learning

A range of problems utilize low-level behaviors that are atomic and repetitive; they involve mastering a “simple” skill like picking up an object or recommending a single product. Problems that involve multilevel reasoning or combine several skills together are much more complex.

In engineering, splitting challenges into smaller components can help to simplify a complex problem. *Hierarchical reinforcement learning* (HRL) uses this idea by decomposing a problem into subtasks, where solving each subtask is easier or better than solving it in entirety. This can lead to computational improvements, greater robustness, more flexibility, and sometimes better performance. It is also one way to mitigate the “curse of dimensionality,” by reducing the search space. For example, when learning to walk, there’s little point searching the action space that includes walking on your hands, unless you’re a gymnast.

With a hierarchy of policies, where only the low-level policies interact with the environment, you can train high-level policies to plan or make strategic decisions over a longer timescale. Another appealing idea is that of encapsulating policies to perform one task well, which you can reuse in other problems; a library of policies.

However, there are a range of challenges that you must consider. How do you define and produce well-encapsulated, low-level policies? How do you train higher-level policies and how do you design the hierarchy? And how do you train all of this in a reasonable amount of time?

Naive HRL

You should consider designing hierarchies manually because in many problems there is a clear separation of concerns. For example, teaching a complex robot to move is clearly distinct from teaching it to make coffee, a rewarding conversation is dependent on the words and utterances used in the conversation, and successful completion of an educational course depends on the content within it.^{14,15} Optimizing these high-level goals through low-level, bespoke skills leads to policies that are flexible, robust, and performant.

In simpler situations it may be preferable to design the abstractions, and therefore the hierarchy, manually. You can then design a bespoke RL implementation that takes advantage of this hand-designed, problem-specific model. [Figure 8-2](#) shows an example of decomposing a notional coffee-making task into a hierarchy of behaviors.

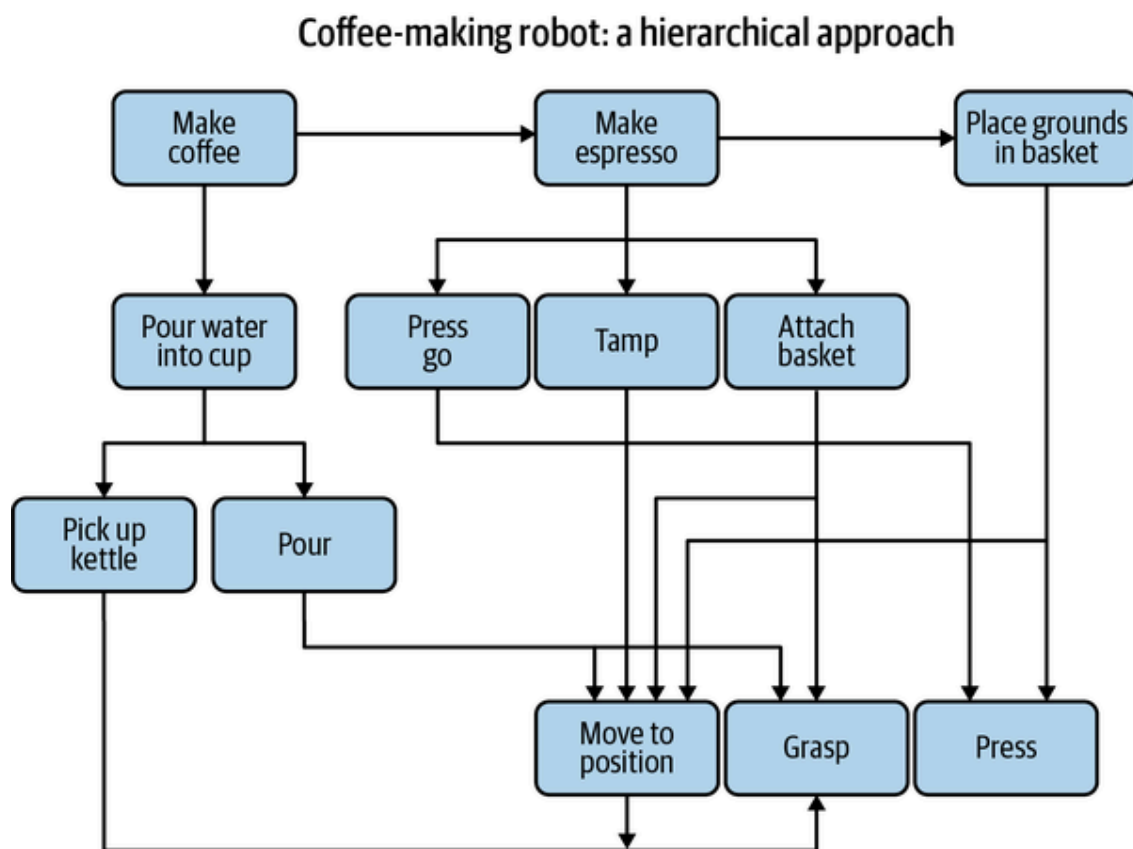


Figure 8-2. You can manually decompose complex tasks into a hierarchy of atomic behaviors and treat each challenge separately. This is an example of decomposing a notional coffee-making task into dependent subskills. Follow the arrows to find the “base” skill.

The major benefit of this approach is that it is likely to be more robust and simpler than a more sophisticated, general approach. It’s a bit like asking a neural network to predict the price of a car from a photo; you will be far more successful building subnetworks that are capable of classifying the make and model, locating the number plate for automated number plate recognition, and so on, and feeding those into a high-level model that predicts price. From a practical standpoint, it also helps you focus on neat, pint-sized problems, rather than one big make-or-break unicorn.

But this approach can only take you so far. In many problems the demarcation of boundaries in the hierarchy is a challenge in itself and an arbitrary line is likely to be suboptimal. When riding a bike, for example, is “pedal” a base skill, or should it be “push down on right foot”? You need a way to automatically find these skills.

High-Low Hierarchies with Intrinsic Rewards (HIRO)

The first step toward an automated HRL solution is to split the policy. A low-level policy is responsible for interacting with the environment through states and actions, as usual, but it does not receive the reward. A high-level policy suggests a high-level action and rewards the low-level policy with a reward that is intrinsic to the requested goal. When the environment produces a reward, it is given the high-level policy. [Figure 8-3](#) presents this idea and is explained in full by Nachum et al.¹⁶

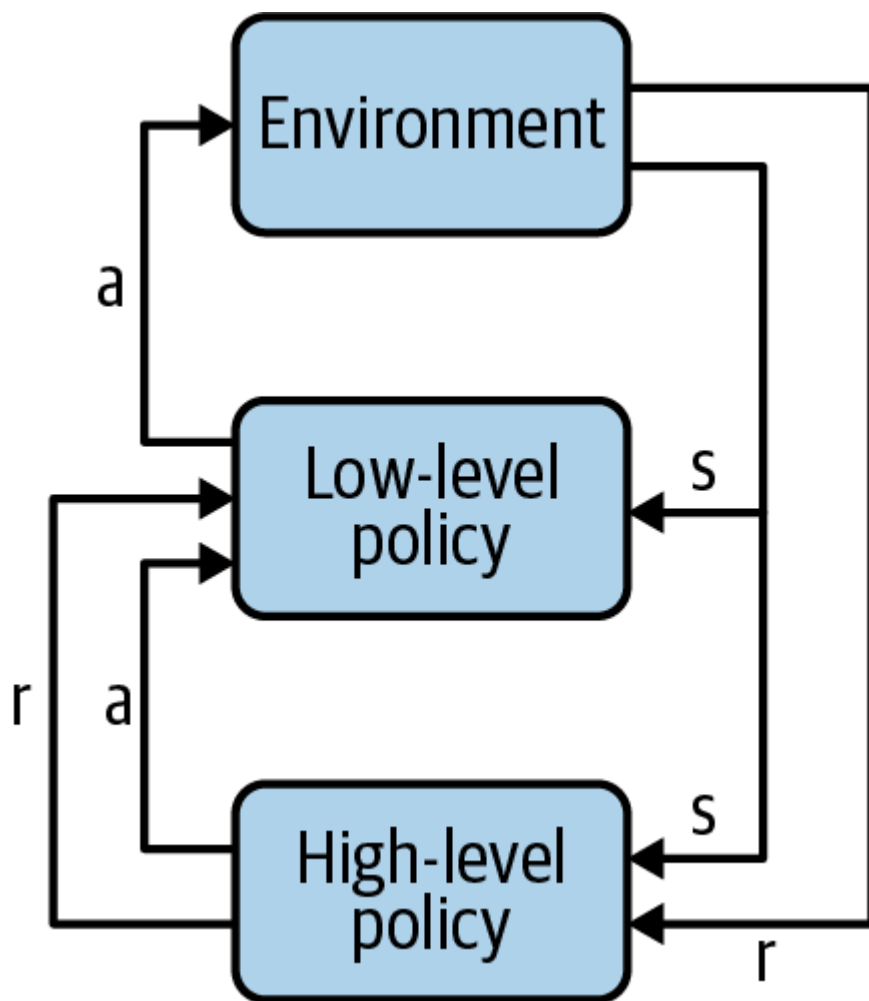


Figure 8-3. Hierarchical RL through intrinsic rewards.

For example, the high-level policy would tell the low-level policy to attempt to move to a specific state, and reward the low-level policy according to how close it gets. The high-level policy samples at a slower rate than the low-level policy to provide a temporal abstraction.

You can parameterize the low-level reward function in any way you like. But researchers propose that a distance-to-state reward helps speed up learning.

The next challenge is that although you can use on-policy algorithms like PPO out of the box, you'd likely want to use a replay buffer to improve sample efficiency. The problem is that the high-level replay buffer will contain observations from previous low-level policies, like right at the start when the low-level policy is random. This means the replay buffer isn't representative and will lead to convergence issues. A simple solution could be to weight the random sample to prefer more recent observations, a form of prioritized experience replay.

But Nachum et al. chose to dynamically *relabel* the high-level action/goals to match the actions that occurred. For example, imagine the coffee example again and the agent has a replay buffer full of the skill "grasp coffee," but the current goal being trained is "push button." In their implementation, the high-level agent sampling from this replay buffer will dynamically change the goal to suit the requested skill. In this example it would alter the goal to be "grasp coffee," to match what is in the buffer.

This might feel like you are fudging the results after the fact to make the current high-level policy look better. But the reward from the environment tends to counteract some of the bias added by the relabeling. In the worst case it is like duplicating observations that are already in the replay buffer. Ideally you want a large selection of uncorrelated, randomly distributed transitions. But if you don't, random sampling helps to make it appear more random.

The [results show](#) that a complex legged simulated robot is capable of both learning to walk and learning high-level tasks such as block pushing, hidden goal finding, and resource gathering.

Learning Skills and Unsupervised RL

A major problem with the previous interpretation of HRL is that this is still a closed-loop system that optimizes for the specific problem at hand. There is no generalization. You couldn't use this in any other situation. It is overfitting the environment.

But you know that a robot arm, for example, moves in a similar way for every task. A command of "move to position x" is specific to the robot and the way it is built, not the environment. All this time I have been describing an agent as if it is implicit to the problem, but actually it is an inde-

pendent entity. So how can you teach policies, or *skills*, that relate to the agent, not the environment?

Researchers propose that agents should be trained to perform a wide variety of skills, even those that don't produce positive rewards. Another higher-level policy can then choose which skills to use to solve a problem. Training an unmanned aerial vehicle (UAV) to do a flip might not be a good thing for a stable communication network, but it might be useful at a [Metallica concert](#).

Diversity is all you need (DIAYN) is a novel idea that promotes the unsupervised discovery of low-level policies. Eysenbach et al. propose that skills should be distinguishable, so they should aim to produce policies that visit different states, and the resulting skills should be as diverse as possible. To encourage randomness you can maximize an entropy measure conditioned on states, actions, or both. [Figure 8-4](#) shows a simplified depiction of this process.¹⁷

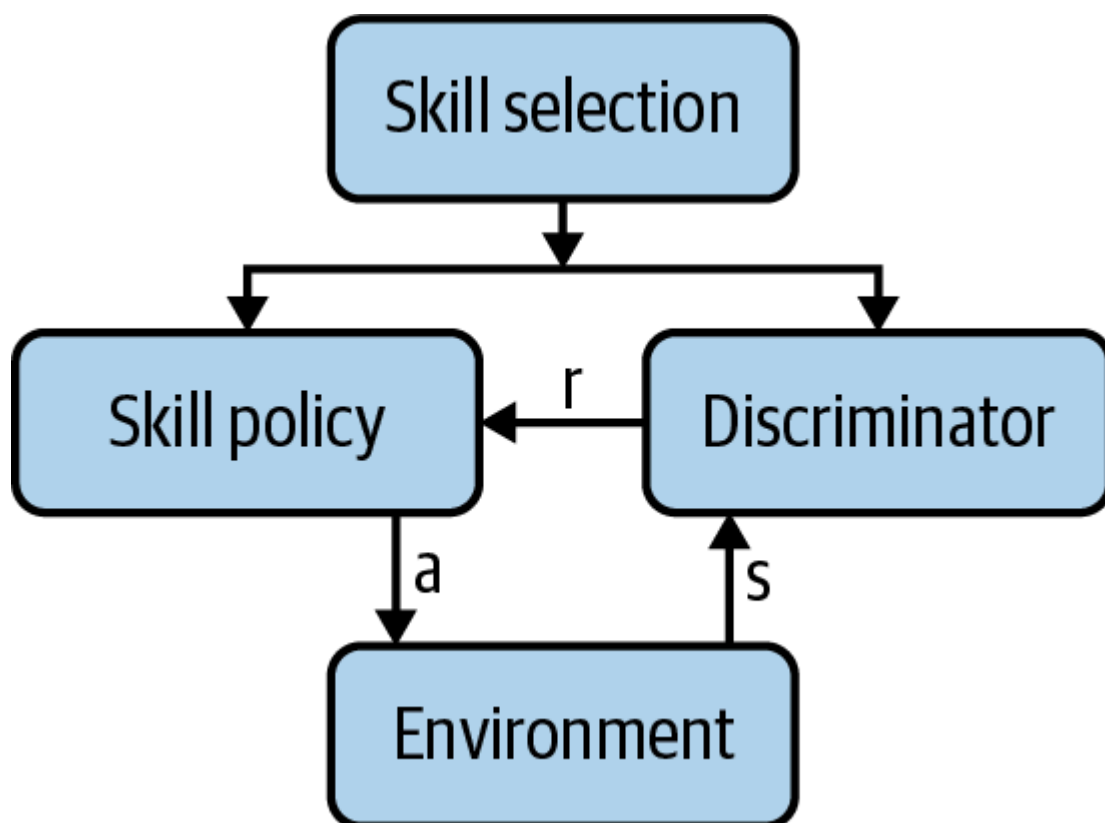


Figure 8-4. Schematic diagram of unsupervised skill learning framework (DIAYN). A skill is chosen at random and a policy acts upon the environment. A discriminator calculates the entropy of the policy, states, and actions that is used as a reward. Over time the policy learns distinct skills.

The [result is a collection of diverse policies](#) that may or may not be useful for a problem. But the diversity acts like a stepping stone to help solve more complicated problems. I particularly like the results from the `HalfCheetah` environment that [show running, crawling, and sitting](#).

Using Skills in HRL

Given a library of skills, how can you train an HRL algorithm to solve a specific problem? I can imagine that pretrained skills could help nearly all RL algorithms by allowing them to concentrate on environment exploration rather than exploring how to explore. This is especially true in environments where there is a significant amount of complexity in the agent itself.

HRL with advantage-based rewards (HAAR) is one algorithm that takes advantage of pretrained skills to dramatically improve initial exploration and therefore speed up learning. Li et al. also introduce the idea of using advantages as the reward signal, rather than a Euclidean distance to state like in HIRO. The reason for this is that the state space isn't necessarily mapped to the real distance to goal. For example, if you are using a video camera, pixel distance isn't the same as physical distance. So instead they propose using the advantage estimate from the high-level policy to re-train and reward the low-level skills. This way, skills that produce a positive advantage, the ones that do well in the high-level task, are promoted over those that don't.¹⁸

Another interesting idea is to anneal the amount of time that each skill is repeated to further encourage long-range exploration.

HRL Conclusions

Building a hierarchy of policies is useful when your problem is sufficiently complex to benefit from splitting. From an engineering point of view, if you can imagine that splitting your policy will help simplify the problem or build more robust abstractions, then HRL might be a good fit. The potential for hierarchical policies is hard to spot. Look for situations where agents have to manipulate complex domains. Robotics is an obvious example, but other domains are just as complex, like agents that deal with text.¹⁹ Conversely, in many real-life problems it is the environment that is complex, not the agent, so HRL may not be beneficial.

In the extreme, HRL is akin to a graph problem, where connections between mini-subpolicies form a large graph. Recent research attempts to

treat state as a graph and use topological features with graph convolutional neural networks to help solve grid-based problems.^{[20](#)}

TIP

Even if you do not work with complex agents, I think that HRL shows that unsupervised learning has an important role to play in RL. And yet it is only just gaining attention from researchers.^{[21](#)} Take the time to consider if your problem could benefit from an initial library of skills or pretraining; it could significantly improve learning speed.

Another interesting question is how much influence do you need over your policies? The skills generated by HRL are not determined by an expert, so they may produce action sequences that are unfamiliar, unexpected, or even unwanted. Of course they are directed by the definition of the unsupervised or supervised reward, so you can influence the skills that way. But an open question, which is probably problem specific, is whether you should interfere to produce human “imagineered” policies or allow the agent to discover its own. I suspect that in most real-life problems it is some combination of both. You don’t want to be overly prescriptive, because that is the whole point of RL—you can’t handcode an optimal policy in complex problems. But you also need to constrain the problem and restrain the agent so that it doesn’t overfit and become brittle.

Multi-Agent Reinforcement Learning

Throughout this text I have considered lone agents. But many problems involve more than one player/agent and it is likely that you can improve performance if you systematically model the problem with multiple agents in mind.

Multi-agent reinforcement learning (MARL) addresses the sequential decision-making problem of multiple autonomous agents that operate in a common environment. Each agent attempts to optimize the long-term return by interacting with the environment and other agents.

MARL applications into fall into three main groups: cooperative, competitive, or a mixture of both. In the cooperative setting, agents attempt to collaborate to optimize a common long-term goal. For example, a cooperative MARL can simulate and optimize a taxation policy.²² In competitive settings, the global reward often sums to zero, since agents can only improve their policy at another agent's expense. This situation is common in domains that include price bidding, like trading and advertising. Finally, the environment may contain both of the previous two situations. For example, an advertisement marketplace wants to maximize both global revenue and long-term user engagement, which means that a cooperative model would be optimal, but advertisers want to pay the least amount of money for the highest impact, which adds a competition into the environment.

The literature generally discusses MARL in the context of games, often traditional games like chess or poker, and increasingly computer games. Like in standard RL, the reason for using games is that they are easy to simulate and verify. Real-life or industrial examples are less common, but are emerging. For example, applications include using MARL to formally define governmental policies, low-level TCP packet congestion mitigation, and multi-robot human-assistance collaboration.^{23,24,25}

MARL Frameworks

The literature tends to model MARL problems in one of two ways, which you can see in [Figure 8-5](#).

Looking at [Figure 8-5\(a\)](#), first recall that an MDP is modeled as an agent that acts upon an environment, which in return produces a new state and a reward. If the environment is episodic then the environment also produces a termination signal.

[Figure 8-5\(b\)](#) illustrates how the first MARL framework is an extension of an MDP called a *Markov game* (MG), where multiple agents operate in parallel. Agents submit unique actions and receive unique rewards, but the state of the environment is global. This is the most obvious extension to an MDP, in the context of RL.

[Figure 8-5\(c\)](#) presents the second and more common model, an *extensive-form game* (EFG), where chance, state, and actions are represented as a

tree, much like the rollout of an MDP. An EFG still alters the state of an environment through actions, even though it is not explicitly shown. For example, if you choose to bet in a poker game then your opponent is forced to match or fold. Your action is impacting the state of the game. But you can see that sequential actions between agents are far more explicit. In [Figure 8-5\(c\)](#), it is clear that agent 2 cannot have a turn until agent 1 has completed theirs. This scenario is not possible with an MG. Note that EFGs are always considered from the point of view of one of the agents.

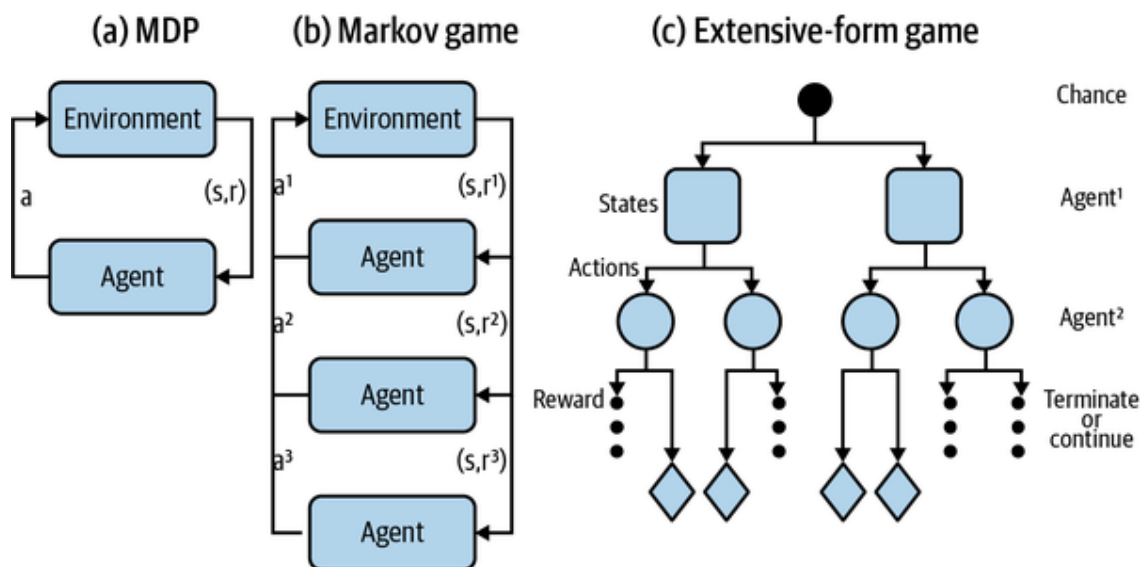


Figure 8-5. Schematic diagrams for different MARL frameworks. (a) represents an MDP, where agents act upon environments that produce states and rewards. (b) is a Markov game where multiple agents have independent actions and rewards. (c) is an extensive-form game, where dots represent chance, squares and circles represent two agents, and diamonds represent terminal states.

Simultaneous actions (like an MG) or imperfect information (like a POMDP) can be represented by *information sets*, which are groups of nodes belonging to a single agent that does not know which node has been reached. These are usually represented by a dotted line or an enclosing circle. You can see an example in [Figure 8-6](#).

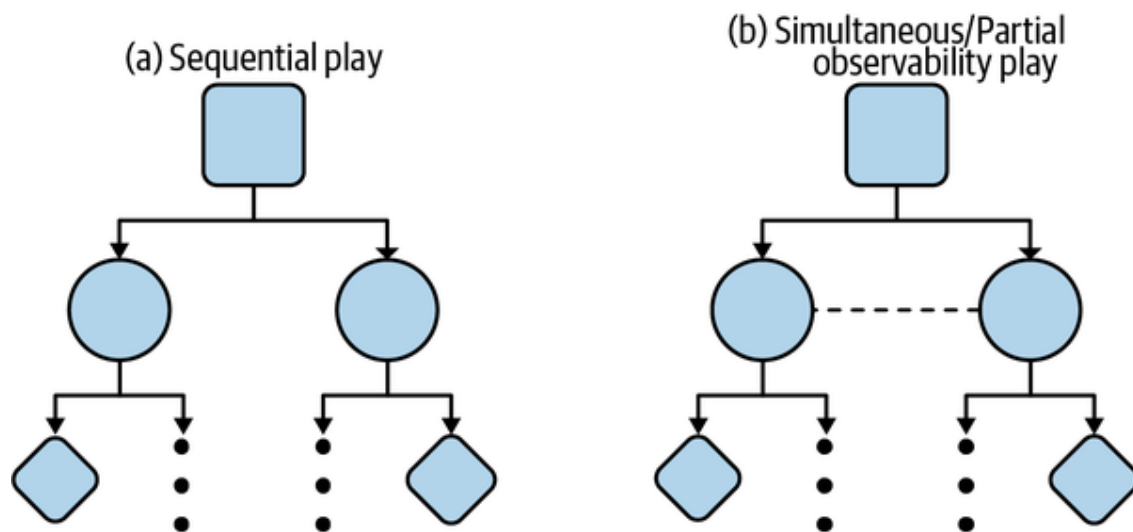


Figure 8-6. Extensive-form game schematics for (a) sequential and (b) simultaneous or imperfect/partially observable games. The only difference is whether other agents know what state they are going to land in.

As an example, consider a game of Rock, Paper, Scissors. This is a classic simultaneous game where two people reveal one of these three actions at the same time to receive the reward. To map this into an EFG you could ask the people to write the actions on a piece of paper, then reveal the choices simultaneously. The agents acted at different times and the second agent could not observe the action of the first player. This means, from the perspective of agent 2, the game is in one of three potential states, but it doesn't know which one.

EFGs are more general and are preferred by theorists. But intuitively, I still like to use MGs as my mental model because the vast majority of industrial situations can be designed to use simultaneous actions and I find the simplicity of a recursive diagram less burdensome than a rolled-out tree. Many researchers agree and use MGs in their applications.

Centralized or Decentralized

Other than the cooperative/competitive distinction, the most important decision to make is whether your problem would benefit from centralized or decentralized learning. [Figure 8-7](#) depicts four scenarios with differing amounts of agent independence.

In centralized learning, or more typically centralized learning and distributed execution, experiences are fed back to a single learner and policies are returned for local execution. The primary benefit of this method is that you can create policies from the experience of multiple actors. For example, if a child could learn centrally, through the experiences of oth-

ers, they would benefit from the large number of experiences and their diversity. Similarly, using the experience of a disparate set of agents allows them to search the state-action space much more quickly. In essence, independent experiences help exploration and centralized learning ensures all agents have similar, viable policies.

In a decentralized model (often called a *DEC-MDP*), both the execution *and* the learning happens locally. This is more scalable because it doesn't matter how many agents you introduce. For example, imagine if everyone "drove" an autonomous vehicle. A single centralized learning process is unlikely to be optimal for all driving conditions. Local learning allows agents to adapt to their local perception of the environment. For example, a policy for driving on the left side of the road is likely to be subtly different than a policy for driving on the right.

Another problem is that agents may have different capabilities. For example, the experience of a cyclist is just as valid as the experience of a car, but combining these experiences in a centralized way is difficult. Robots may have different configurations. Decentralized learning allows agents to learn to use the actions at their disposal.

A further benefit is that the decentralized model potentially allows you to use standard single-agent RL algorithms, because you can treat each agent independently.

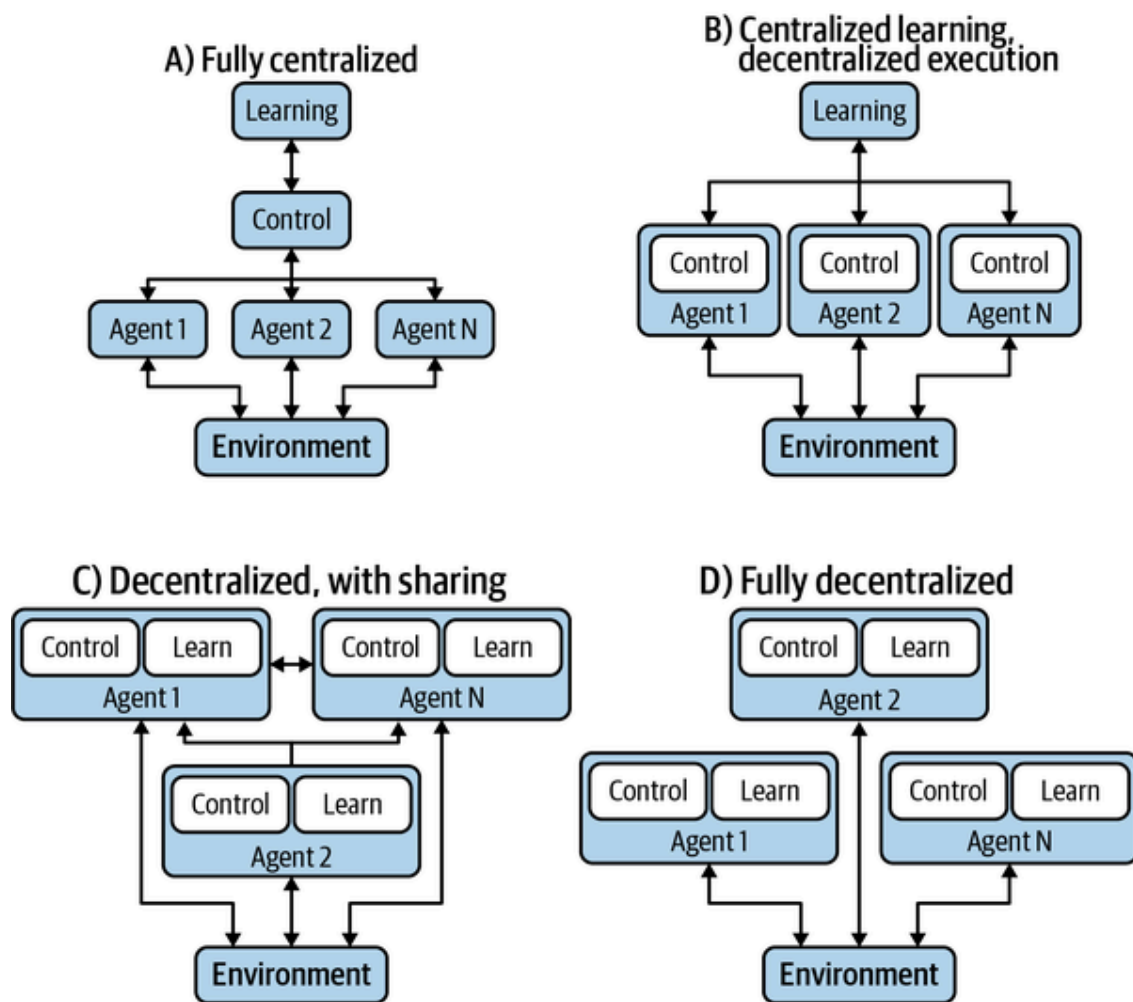


Figure 8-7. Different forms of control and learning in MARL. (a) In fully centralized settings, agents have no control over execution or learning. (b) In centralized learning, agents have full execution control but policies are distributed by a central source. (c) In a decentralized setting, if communication is possible, then agents can share experience and policies. (d) In a fully decentralized model, agents are independent.

Single-Agent Algorithms

The first and most obvious route to solving MARL problems is to assume each agent is independent and apply any of the single-agent RL approaches discussed in this book. Unfortunately, traditional RL algorithms such as Q-learning or policy gradient are poorly suited to multi-agent environments. The primary concern is that during training other agents are interacting and altering the environment state. This makes the environment nonstationary from the perspective of any single agent. When the environment is constantly changing, this impedes the agent's ability to create a stable policy. This also means that common RL add-ons, like basic experience replay, do not help. This does not mean that they fail completely, however. The agent can learn stable policies in simple environments, but they are usually suboptimal and tend not to converge when environments increase in complexity.

It is technically possible to use model-based policy gradient algorithms, because the model provides you with a perfect description of the dynam-

ics of the environment, but for many problems a model is not available.

Case Study: Using Single-Agent Decentralized Learning in UAVs

Using UAVs, often called drones, is becoming increasingly common in applications where geospatial coverage is important but hard to achieve. UAVs can fly into an unknown environment to monitor wildfires, perform search and rescue, and establish communication networks.^{[26,27,28](#)} Their popularity is due to the low operational cost, the array of sensors, and their flexibility of application.

It is impractical to attempt to handcode controllers to perform tasks due to the lack of an accurate model. RL provides a natural model-free solution that is capable of navigating through difficult environments. There are a variety of examples of using RL to learn low-level UAV primitives like flying, moving, and avoiding obstacles. But UAVs shine when they are used in collaboration to solve a problem.

Cui, Liu, and Nallanathan use a simulated fleet of UAVs to provide wireless infrastructure to regions with limited coverage. UAVs are particularly useful in this situation because it is typically easy to establish line of sight. The researchers use MARL to solve a complex optimization problem that maximizes network performance.

Their work assumed agent independence and leveraged a simple single-agent solution. They used Q-learning to learn the optimal choice of user to connect to, subchannel, and power level. Note that positions are fixed in this work. They develop a reward function based upon quality metrics and solve in a simulation of three UAVs. Note that there is no correction for the nonstationarity of the environment and agents must compete for reward.

So there is a lot of room for improvement. A better decentralized technique could account for the instability of the environment. Or since they have a network, a centralized technique would make development easier. And a more sophisticated agent could handle a much more complex state space, like spatial positioning, which would help the solution scale. With these fixes it should be feasible to run this in real life. The problem with

the way that research is celebrated today is that it only promotes success, not failure. It's likely that Cui, Liu, and Nallanathan tried some of these ideas but ran out of time.

Despite this I am optimistic and can imagine a more stable set of algorithms that are able to learn more complex behaviors (like optimal multi-agent positioning).

Centralized Learning, Decentralized Execution

Recent successes have arisen by combining improvements in RL with the sample efficiency of centralized learning. One such algorithm that gained notoriety combines DDPG ([“Deep Deterministic Policy Gradients”](#)) with a centralized learning, decentralized execution approach called *multi-agent DDPG* (MADDPG).²⁹

DDPG (or TD3/PPO/others) provides the backbone that allows agents to learn policies without a model of the environment. Policy gradient algorithms provide a theoretical basis for combining the gradients of multiple agents, as opposed to value methods in which the global action-value function is not representative of local situations. Using a centralized learning approach during training time improves sample efficiency and speeds up learning. During evaluation, agents use independent policies and local observations.

The primary motivation for this approach is that if you know the actions of all agents, then it is possible to predict or track the changes to the environment. Like in quantum mechanics, if you know the state and actions of all atoms in the universe, then life is stationary, and you can predict the future (or past).

To calculate the value of a policy for an individual agent, you need to know the outcome of the agent's policy and the value predicted in the environment. But the environment is affected by all other agents, so you need to provide information about the actions of all other agents. Looking back at the different policy gradient implementations ([Equation 5-11](#)) it is clear that the actor-critic paradigm naturally considers the actions of an agent, so all you need to do is extend it to consider the actions of *all* agents, which is shown from the perspective of a single agent in [Equation 8-1](#), with N other agents in the environment.

Equation 8-1. Multi-agent actor-critic

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi_{\theta}} \nabla_{\theta} \ln \pi_{\theta}(a | s) Q_w(s, a) && \text{(Actor-Critic)} \\ &= \mathbb{E}_{\pi_{\theta}} \nabla_{\theta} \ln \pi_{\theta}(a | s) Q_w(s, a_1, a_2, \dots, a_N) && \text{(Multi-Agent Actor-Critic)} \end{aligned}$$

This leads to the idea that we can have a local, independent actor but a centralized critic. Interestingly, because there is an actor and critic learned for every agent, agents can have different policies and rewards, which means that this works in both cooperative and competitive settings. Next, and I leave this as a homework exercise, you can apply the various improvements to vanilla policy gradient algorithms to end up with multi-agent versions of standard actor-critic RL algorithms.

Most actor-critic algorithms need the policy to generate the next action for the training of the action-value function. See any of the actor-critic algorithms, for example. One simple solution is to ask every agent to pass the policy, or all of the next actions (depending on the algorithm), back to the centralized critic. This causes synchronization and latency issues, which hampers efficiency, but is quite simple.

The approach that MADDPG took was to *learn* the policies of the other agents, while they themselves are learning. This is an obvious source of nonstationarity. To limit the effects of nonstationarity, MADDPG slows down the learning rate by updating a limited subset of random policies. Not ideal, but it works. See the paper for more details.

Decentralized Learning

If you can't use a centralized model, then you need ways in which to stabilize decentralized learning. Temporal-difference hysteresis is a technique that uses two different learning rates depending on the size of the temporal-difference error.³⁰ If the temporal-difference error is below a threshold usually set at zero, which means that the agent obtained less reward than expected, a slower (smaller) update rate is applied. It assumes that the bad return was because the agent was exploring—you wouldn't want your child to copy you if you do something stupid. When the error is positive, the agent uses the normal learning rate. In essence it is biasing the algorithm to be more optimistic, which itself can lead to issues, but that can often be controlled by the learning rate hyperparameters. The key, however, is that it helps with the nonstationarity of the environment;

the agent doesn't revert a potentially good policy just because of a few lousy updates that were due to a changing environment.

The second challenge in decentralized learning is partial observability. The classic and simplest solution is to include a recurrent layer to record histories, although other approaches are available. I anticipate that more formal approaches are just around the corner.

The final challenge is the inability to use standard replay buffers. The problem is that experiences are local and nonstationary when sampled independently, despite agents learning concurrently. The local view at that point in time is unlikely to represent the environment at a future point in time. A related issue is that randomly sampling from different parts of the replay buffer may have entirely different environmental states, which in the worst case could cause two agents with exactly the same local conditions resulting in completely different policies. There are many potential ways to improve upon this, but Omidshafiei et al. implemented a replay buffer that was synchronized over all agents. Concurrent experience replay tends to lead to local correlations, which is typically a bad thing and resolved by random sampling, but in this situation helps because it forces all agents to focus on the current replay, which should all have the same environmental conditions. For example, if two agents had similar local conditions, then sampling from the buffer at the same point should yield similar experiences and therefore lead to similar policies. In this particular instance the experiences are trajectories in order to train the recurrent neural networks. This combination of DQN, hysteresis, and synchronized replay buffer is called Dec-HDRQN.³¹

Note that this is just one of many potential ways to solve the problem of nonstationarity during updates, different local optima during experience replay, and partial observability. You could swap out any of the previous components with other implementations, although they will have their own idiosyncrasies.

Other Combinations

The centralized and decentralized paradigms can be mixed together in a huge variety of ways to suit the application. For example, you could use a centralized critic, with a centralized policy that is shared with agents for remote execution. Or you could use agents with decentralized critics and

policies that are shared with their immediate neighbors. Many permutations have been investigated and there is likely to be an algorithm that suits your use case. A detailed investigation of this varied field is outside of the scope of this book, but there are some great resources in [“Further Reading”](#).

For me, MARL highlights the importance of the problem design. The sheer number of permutations, combined with the number of RL algorithms, combined with the number of neural network possibilities, means that it is computationally difficult to automate or perform a large-scale hyperparameter scan. You have to choose a small subset of algorithms and architectures to make the feedback cycle small enough to produce anything in a reasonable amount of time. This means that it is vitally important to have a correct and robust problem definition; otherwise, if you make a small change to the problem, this can have huge impacts on the implementation. In the worst case you might have to throw away all your work and all your research and start again.

Imagine you had a problem where you decided that centralized training was possible. You spent many weeks doing your research, building simulators and testing lots of algorithms. Your initial tests were performing well and you told your stakeholders that you are ready to move to the next step. Then you start testing in the real world on a much greater number of agents. You leave for the day happy and sleep well.

In the morning you turn on your laptop, excited to see what new policies have been found. But you realize that (a) all policies are no better than random and (b) there have only been four training steps. After denial, anger, “quick fixes,” and depression, you realize that most of the time the agents weren’t able to communicate with the server and your code waits until every agent has checked in before it runs a training step. It quickly dawns on you that in real life agents do not have reliable communication and even if they did, you will eventually have too many agents, which will slow down the whole system. You need a decentralized model.

The only option is to report a failure to your stakeholders and go back to the start. I wouldn’t call it a complete waste of time, because this experience allowed you to learn a lot of transferable knowledge and skills, but in my experience, stakeholders generally don’t like moving backward on a product roadmap.

This pattern of failure, an incorrect problem definition, is common in data science, too. But in RL there tends to be a much higher barrier to experimentation because of the agent–environment interaction, so the amount of time wasted can be much greater.

TIP

One mitigation, among others discussed throughout this book, is to take your time when defining the problem and subsequent modeling. Your model needs to be simple enough to solve and reason about, but it must also be representative of the real problem. It’s like software: the best code is simple and obvious, yet seems to handle the majority of situations. But don’t think that simple means easy; simplicity arises out of effort, trial, and many errors.

Challenges of MARL

Despite significant progress and a daunting amount of research, MARL is hard. This doesn't mean that it is impossible and I don't want to dissuade you from going down this path; the research shows that solutions are entirely doable. But I want to make you aware of the challenges to watch out for, which include:

Diversity of problems

You need to accurately model your environment, making decisions about cooperative or competitive, sharing policies or not, sequential or simultaneous, and you also need to consider all of the normal MDP questions like value or policy or fully observable or partial. Once you have a solid model, then you need to dig to find an appropriate algorithm that works within the assumptions of your model. MARL is a much more complex problem, so there are far more potential solutions.

Multidimensional and multimodal goals

The goal of one agent may be significantly different than another, due to the local conditions of the environment. This can make it challenging to share policies.

Scalability

In many tasks it is important to synchronize information, whether that be external, like the current state of an agent, or internal, like the current policy, which leads to communication efficiency issues in larger deployments.

Instability

Since agents are actively altering the environment, from the perspective of another agent, the environment is nonstationary. This breaks or invalidates some of the basic assumptions provided by the MDP and leads to lack of stability.

Optimality

It's hard to precisely define optimality for all but the simplest of examples, which means there are no convergence guarantees for

nontrivial environments.

These challenges are unique to MARL, but they sit on top of all the standard challenges in RL like partial observability and sample efficiency. It is likely that given time these challenges will reduce in severity as researchers find different ways of tackling RL.

MARL Conclusions

MARL is an exciting and rapidly developing extension of RL. I anticipate that significant improvements are just around the corner. In the month of June 2020, for example, there were more than 50 new MARL research papers on arXiv. But even now it is possible to leverage MARL to solve a variety of commercially viable use cases, some of which you can see on the [accompanying website](#).

In this brief overview of MARL I have ignored most of the game-related literature. It is still very important from a research and theoretical perspective, but out of the scope of this book.

Expert Guidance

In many of the examples or situations presented throughout this book I have talked about teaching an agent a policy from scratch. But humans possess domain knowledge that can direct, encourage, and improve how agents learn. For example, the age at which children start school varies by country. In the US and United Kingdom, kids start around the age of four or five, but in many countries like Serbia, Singapore, South Africa, and Sweden, children don't start until they are seven. The argument for starting later is that children are given more time to randomly explore, to fine-tune low-level emotional and motor skills before they are forced to learn arbitrary high-level abstractions. Whereas the counterargument is that children waste time with random exploration and that it's better to subtly guide them using a curriculum. The same is true in RL; expert guidance is likely to provide valuable information, but the agent also needs to explore by itself. There isn't a correct amount of either.

But I am certain when saying that problems can benefit from some amount of expert tutelage. This section introduces how to incorporate

this expertise into your policy.

Behavior Cloning

A traditional approach to incorporating expert policies is to use supervised learning to maximize the likelihood of an agent choosing actions that are the same as the expert for encountered states. This is called *behavior cloning* but isn't popular because it overfits the expert demonstrations. If an agent were to find itself in a state that wasn't demonstrated, then the agent wouldn't know what to do. This situation, because of high-dimensionality in complex problems, is very likely to happen, which makes it important to use algorithms that continue to learn from their actions, even if they aren't demonstrated.³²

Imitation RL

In 2017, Hester et al. proposed that in many real-life situations an accurate simulation is not available or feasible; the only possibility is to train the agent in situ. But there are external sources that already use viable policies, like a previous controller or human guidance. Developing solutions to transfer knowledge from these experts to agents enables a range of solutions for problems where demonstrations are common but accurate simulators are not.³³

First, an expert (human or machine) generates trajectories in the environment. Then a supervised loss is incorporated into the Bellman update that measures the difference between the actions of the expert and the agent. Over time the agent learns to generate trajectories similar to the expert.

The problem with this method is that you don't want the agent to mimic the expert, since the expert's policy is likely to be suboptimal. *Deep Q-learning from demonstrations (DQfD)* accounts for this by attempting to prevent overfitting to the expert trajectory through a series of regularizations. *Generative adversarial imitation learning (GAIL)* suggests a similar approach that adversarially perturbs action distributions toward an expert. But this doesn't answer or have an intuitive knob to tweak that controls *how* much expert advice to use.³⁴

Soft Q imitation learning (SQIL) provides one idea and at the same time simplifies DQfD and GAIL by changing the way standard algorithms use

experience replay. Reddy et al. found that (a) carefully controlling the trajectories that reside in the replay buffer, (b) reconstructing the reward signal for those trajectories, and (c) using a loss function that allows exploration but stays close to expert trajectories allows the agent to learn from experience even when there is no extrinsic reward signal. The resulting algorithm is broadly similar to behavior cloning, except that it allows for deviations into states that were not demonstrated. It penalizes those states through regularization and nudges the agent back toward expert behavior. You could even control the amount of expert advice to use by weighting the regularization. So I can imagine an annealing version that initially uses expert demonstrations and tends toward pure RL over time. But the main selling point is the simplicity, and this can be adapted to any off-policy algorithm, discrete or continuous.³⁵

Inverse RL

The problem with imitation RL is that there is no guarantee that the expert is demonstrating an optimal policy. For example, imagine you were training your car to “drive well” and to help your algorithm you recorded your driving and submitted that as an expert demonstration. Are you really an expert driver? Even if you are, should the agent be learning from your biased view of driving?

Another approach is ignoring the trajectories and concentrating on the goal. Producing a reward for “driving well,” however, is hard to quantify. You could suggest that you want to get to your destination in the shortest time possible and use total elapsed time as a reward, but this ignores getting there safely. You could iteratively establish all of the main concerns but then how do you weight them? In general, it is difficult to design rewards for complex domains so instead, why not learn the reward?

Inverse RL (IRL) attempts to predict what a reward would look like, given demonstrations from an environment.³⁶ The general aim is to find an arbitrary reward function, based upon demonstrated trajectories, that explain expert behavior. Designing reward functions is a major challenge in RL, and IRL promises to replace the manual specification with demonstrations, which are arguably less biased and easier to specify. It is also argued that learning from demonstrations is more robust because they don’t make any assumptions about the underlying environment.

Most IRL methods fit the following pattern:

1. Assume the expert's demonstrations as a solution to an MDP
2. Use a parameterized model as the reward function
3. Solve the MDP with the current reward function
4. Update the reward function to minimize the difference between the expert's trajectory and the learned trajectory (or policies if those are available)
5. Repeat until stable

Figure 8-8 depicts this process for a simple 3×3 discrete grid. An RL agent is trained to solve MDP with a parameterized reward function. Each time the MDP has learned an optimal policy it is compared to the expert trajectories. If the learned policy visits a state that is also visited by the expert, the reward for that state is increased (or equivalent reward parameters are changed). Over time the reward function tends toward the expert trajectory.

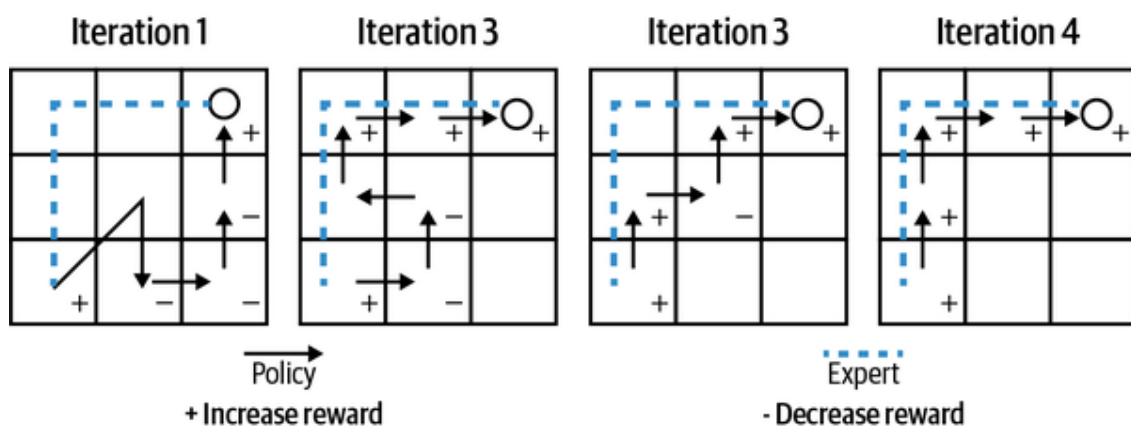


Figure 8-8. Example iterations of a notional IRL problem. The environment is a 3×3 grid. A single expert trajectory is provided (dashed lines). The policy (solid lines) uses a parameterized reward function in the MDP. Over time the reward function is altered to be more likely to produce expert trajectories. Eventually you end up with a reward function that mimics expert behavior.

Most of the problems with IRL stem from the fact that the expert trajectories are likely to be noisy and suboptimal. Researchers have developed a wide range of algorithms that adapt this basic process to improve robustness against noise (see the entropy-based or probabilistic IRL algorithms) or to loosely direct learning toward optimality, rather than matching the expert trajectory, which tends to be more problem specific.

Another major problem is that these methods expect you to repeatedly retrain RL algorithms, which as you know can take a long time. This means IRL doesn't scale to anything but the simplest of state spaces. All is not

lost, though; you can use a dimensionality reduction technique like autoencoding or hashing to reduce the high-dimensional state into something that IRL can handle. ^{37,38}

One final remark I want to make is that when researchers talk about expert guidance in their papers, they tend to assume that the experts are successful at what they do. Failure is far more common than success, and there is no reason why you can't include disastrous trajectories in IRL; just demote the rewards for these trajectories instead. Shiarlis et al. suggest that generating failed demonstrations is even easier than generating good ones. ³⁹

Curriculum Learning

When you learn science, sailing, or snowboarding, there is a well-defined series of steps or tests that you have to achieve before you can move onto something more advanced. In snowboarding you first train your leg muscles how to control the edge, then you train your core to shift your weight to turn, and finally you learn how to twist in order to carve and spin. In science you first learn about atoms before fusion, Newton's laws of motion before fluid dynamics, and cell structures before DNA. These are curricula, high-level meta-learning guides that show you common strategies to learning advanced policies.

Curricula can be used in RL to help reduce the amount of exploration an agent needs to perform. You can specify a series of simpler subgoals that lead to learning the main goal. This sounds very similar to "Imitation RL" and several techniques under the curriculum banner use imitation techniques to derive curriculum subtasks.

Markov games (or other MARL frameworks) are important use cases for curriculum learning. You can often alter the formulation of a problem to play agents against each other in *self-play*. The most obvious form of self-play is in the highly publicized board game examples, but researchers have applied the same idea to slightly more realistic environments like mazes or robotic movement. ^{40,41} There are many options for how to design the interaction in self-play but one common form is to use one instantiation of an agent as a teacher and another as a student. The teacher suggests "tasks" by performing them in the environment (and therefore generating a trajectory) and the student solves that task by trying to per-

form the same “task.” I say “task” because in practice they are just moves to a particular state, which may or may not translate to something humans would consider a task. Rewards are shaped to encourage the student to tackle more complex tasks over time by increasing the path length of the teacher. Another common form is to incorporate imitation-like techniques to automatically classify skills and repeatedly ask the agent being trained to practice skills.⁴²

Another interesting idea that is similar to both imitation and curriculum learning is the combination of policies. *Residual RL* defines a way of combining two policies to solve a single task. It’s not presented as hierarchical RL but could also be considered as such. The idea is that if you have an accurate model for part of the task, which is common in robotics, for example, then you can combine that policy with another to achieve tasks that are much harder to model like grasping or touching. This is introduced through the addition of policies. I like this idea because it is so simple. You have seen many examples of altering the reward function or the loss function by arbitrarily adding bonuses or penalties, but this is the first example of combining actions. Intuitively, I think there is more to this than first meets the eye. I can imagine a more general action-altering framework that is applicable to other areas of RL (safe-RL, for example).⁴³

Curriculum learning is another hot topic because it is one avenue that could potentially help RL and ML to scale to really complex problems. The idea that you can engineer your problem so that there is an unsupervised solution to learn a range of important skills or possibly even solutions is enticing. For more information see the recent reviews in [“Further Reading”](#).

Other Paradigms

Researchers like to coin a term. This can lead to a deluge of other paradigms that can be hard to interpret. Often some of them will break out into popular parlance, like Q-learning, whereas others could be big in the future but might not be, like meta-learning. I introduce this here along with transfer learning because they didn’t quite fit anywhere else. They are big topics in their own right, but I consider them to be quite advanced and therefore a detailed discussion is out of the scope of this book.

Meta-Learning

Meta-learning, or learning-to-learn, aims to develop a general framework that optimizes the performance of learning. In other words, if you knew the best way to learn, you derive an algorithm that is optimal for your task.

One of the most important applications of meta-learning is quantifying or improving learning performance when there are few observations or rare events. Humans are capable of learning patterns and policies from small numbers of observations by transferring knowledge from past experience or other domains.

The problem with *meta-RL* is that it is so abstract there are many ways in which you could imagine improving learning performance. For example, some implementations implement artificial curiosity and boredom.^{[44](#)} Others use genetic, evolutionary, or adversarial approaches.^{[45](#)} Some even combine imitation and inverse RL to the mix.^{[46](#)}

All this noise makes it hard to define precisely what meta-RL is or where it is heading. If the previous research is anything to go by, then meta-RL will continue to be a mash-up of RL and ML techniques with the goal of solving problems with less data, faster.

Transfer Learning

Transferring the experience of one agent to another is of obvious interest. If you can train a robot in simulation and then optimally transfer that knowledge to the real world, you can artificially increase the sample efficiency of your RL algorithm. Historically, transfer learning was considered separately, but today it tends to be absorbed into meta-learning. It is still important for industrial applications, however.

A common problem when converting research-oriented RL algorithms to industrial applications is that nearly all research papers train from scratch. But most of the time similar problems have already been solved, as demonstrated in the literature or by your previous work. It makes no sense to throw away all that experience and the resulting policy.

Like meta-RL, however, there are many ways in which you can reincorporate past experience into new models. You could reuse raw experiences via replay buffers, copy/paste autoencoders, use previous policies to help guide the new one via curriculum learning or concoct some kind of cooperative MARL solution. The possibilities are endless and there is no right way.

Fundamentally there are three types of transfer. The first, and most common, is to attempt to transfer policies that are used for the same task in the same domain. Pre-training with a simulation is an example of this. The second is transferring a policy in the same domain to achieve a slightly different task. This is the goal of curriculum learning and common in applications where a single agent is capable of solving multiple tasks, like robotics. The third is transferring policies across domain boundaries, where there may be different states and actions. This happens often accidentally due to partial observability, but a classic example would be attempting to train an RL algorithm to play the Atari game *Pong*, then get the same policy to play *Breakout*.

I think this is still an important subject for industrial applications, because it allows you to build solutions and train algorithms in a more sample-efficient way, which is often the bottleneck in real applications. But it is beyond the scope of this book, so if you are interested, I provide some starting points in [“Further Reading”](#).

Summary

The purpose of RL is to learn optimal strategies, so it makes perfect sense to include as much extra information as you can to help solve your problem better and quicker. The first step on this journey is to ensure that your basic assumptions are valid and to check if your problem definition can be improved. This often starts by questioning the validity of the MDP and leads down new avenues of potential solutions. The classic example is that many real-life domains are partially observable, but are rarely modeled as such. Accepting the partial observability can lead you to solutions that happily handle uncertainty.

Hierarchical RL (HRL), especially skill-aware RL, has the potential to significantly scale industrial solutions. If original equipment manufacturers

or vendors can derive and reuse “skills,” this might help them provide more solutions using the same processes, hardware, and software. This will help open up new use cases and help diversify revenue streams. For these reasons I believe that HRL has a very important part to play in industry.

Multi-agent RL (MARL) is a very popular research topic because it presents a good way to solve distributed problems. For example, bush fires are increasing in both quantity and intensity, so having intelligent solutions for geospatial problems like this can help in that fight.⁴⁷ On a technical level, too, it makes sense to tackle the exploration problem by using the experience of multiple agents in the same environment, perhaps cooperatively. And competitive scenarios arise in a number of domains, especially in those that involve money.

Utilizing expert advice to guide strategies is another pattern that doesn’t need explaining. One of the main issues in complex industrial problems is constraining the problem definition and therefore the exploration space. Industry expects and demands more performance and improved efficiencies and one of the best ways to get there is to bootstrap, just like *n*-step methods did way back in [Chapter 3](#). On a strategic level, bootstrapping is essentially a process of copying successful strategies, whether that be from expert guidance or another model. Once you have extracted that expertise then you can work on fine-tuning it with standard RL techniques.

And finally, RL is a rapidly expanding topic and as abstractions become higher and higher they tend to veer off in weird and wonderful directions. Some of these will become the next big thing, others will sit quietly in a corner. I think that the use of meta-learning in RL, for example, will become more important as engineers gain more practical experience with RL. I think we’re in the *what* phase now and soon engineers will be asking *why*. With adoption that will turn toward *how* and meta-RL presents a set of ideas to answer that question. But as the name suggests, the topic is very philosophical and can be hard to qualify. I’m sure that will improve with time, if indeed that is the right direction to head in.

Further Reading

- Hierarchical reinforcement learning:

- [Great overview video of the technical difficulties by Doina Precup.](#)
- Multi-agent reinforcement learning:
 - Schwartz, H. M. 2014. *Multi-Agent Machine Learning: A Reinforcement Approach*. John Wiley & Sons.
 - A great RL-focused recent review from Oroojlooy and Hajinezhad.⁴⁸
 - Another review that spans games and EFGs from Zhang et al.⁴⁹
 - Decentralized MARL:
 - A great [introduction to decentralized MARL from Christopher Amato](#), and accompanying book by Oliehoek and Amato.⁵⁰
 - Expert guidance:
 - A recent survey of inverse RL.⁵¹
 - Curriculum learning:
 - Two recent surveys.^{52,53}
 - [Lillian Weng's blog](#) is always a great resource and has a nice overview on curriculum learning.
 - Meta-RL:
 - Huimin Peng has bravely created a good review on meta-RL.⁵⁴
 - Transfer learning:
 - There's a great chapter by Alessandro Lazaric that develops transfer learning at a high level.⁵⁵
 - An interesting recent review of transfer learning in general (not RL-specific).⁵⁶

- ¹ Karl, Maximilian, Maximilian Soelch, Justin Bayer, and Patrick van der Smagt. 2017. [“Deep Variational Bayes Filters: Unsupervised Learning of State Space Models from Raw Data”](#). ArXiv:1605.06432, March.
- ² Hefny, Ahmed, Zita Marinho, Wen Sun, Siddhartha Srinivasa, and Geoffrey Gordon. 2018. [“Recurrent Predictive State Policy Networks”](#). ArXiv:1803.01489, March.
- ³ Wayne, Greg, Chia-Chun Hung, David Amos, Mehdi Mirza, Arun Ahuja, Agnieszka Grabska-Barwinska, Jack Rae, et al. 2018. [“Unsupervised Predictive Memory in a Goal-Directed Agent”](#). ArXiv:1803.10760, March.
- ⁴ Qiao, Zhiqian, Katharina Muelling, John Dolan, Praveen Palanisamy, and Priyantha Mudalige. 2018. [“POMDP and Hierarchical Options MDP with](#)

- [Continuous Actions for Autonomous Driving at Intersections](#)". 2018 21st International Conference on Intelligent Transportation Systems (ITSC), 2377–82.
- 5** Qiao, Zhiqian, Zachariah Tyree, Priyantha Mudalige, Jeff Schneider, and John M. Dolan. 2019. ["Hierarchical Reinforcement Learning Method for Autonomous Vehicle Behavior Planning"](#). ArXiv:1911.03799, November.
 - 6** Hochberg, Irit, Guy Feraru, Mark Kozdoba, Shie Mannor, Moshe Tennenholtz, and Elad Yom-Tov. 2017. ["A Reinforcement Learning System to Encourage Physical Activity in Diabetes Patients"](#). *Journal of Medical Internet Research* 19 (10): e338.
 - 7** Hallak, Assaf, Dotan Di Castro, and Shie Mannor. 2015. ["Contextual Markov Decision Processes"](#). ArXiv:1502.02259, February.
 - 8** Ie, Eugene, Chih-wei Hsu, Martin Mladenov, Vihan Jain, Sanmit Narvekar, Jing Wang, Rui Wu, and Craig Boutilier. 2019. ["RecSim: A Configurable Simulation Platform for Recommender Systems"](#). ArXiv:1909.04847, September.
 - 9** Chandak, Yash, Georgios Theodorou, Blossom Metevier, and Philip S. Thomas. 2020. ["Reinforcement Learning When All Actions Are Not Always Available"](#). ArXiv:1906.01772, January.
 - 10** Boutilier, Craig, Alon Cohen, Amit Daniely, Avinatan Hassidim, Yishay Mansour, Ofer Meshi, Martin Mladenov, and Dale Schuurmans. 2018. ["Planning and Learning with Stochastic Action Sets"](#). ArXiv:1805.02363, May.
 - 11** Chandak, Yash, Georgios Theodorou, Blossom Metevier, and Philip S. Thomas. 2020. ["Reinforcement Learning When All Actions Are Not Always Available"](#). ArXiv:1906.01772, January.
 - 12** Chandak, Yash, Georgios Theodorou, Chris Nota, and Philip S. Thomas. 2020. ["Lifelong Learning with a Changing Action Set"](#). ArXiv:1906.01770, May.
 - 13** Li, Xiang, Wenhao Yang, and Zhihua Zhang. 2019. ["A Regularized Approach to Sparse Optimal Policy in Reinforcement Learning"](#). ArXiv:1903.00725, October.
 - 14** Saleh, Abdelrhman, Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, and Rosalind Picard. 2019. ["Hierarchical Reinforcement Learning for Open-Domain Dialog"](#). ArXiv:1909.07547, December.
 - 15** Zhou, Guojing, Hamoon Azizsoltani, Markel Sanz Ausin, Tiffany Barnes, and Min Chi. 2019. ["Hierarchical Reinforcement Learning for Pedagogical Policy Induction"](#). *Artificial Intelligence in Education*, edited by Seiji Isotani, Eva Millán,

- 16** Nachum, Ofir, Shixiang Gu, Honglak Lee, and Sergey Levine. 2018. [“Data-Efficient Hierarchical Reinforcement Learning”](#). ArXiv:1805.08296, October.
- 17** Eysenbach, Benjamin, Abhishek Gupta, Julian Ibarz, and Sergey Levine. 2018. [“Diversity Is All You Need: Learning Skills without a Reward Function”](#). ArXiv:1802.06070, October.
- 18** Li, Siyuan, Rui Wang, Minxue Tang, and Chongjie Zhang. 2019. [“Hierarchical Reinforcement Learning with Advantage-Based Auxiliary Rewards”](#). ArXiv:1910.04450, October.
- 19** Saleh, Abdelrhman, Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, and Rosalind Picard. 2019. [“Hierarchical Reinforcement Learning for Open-Domain Dialog”](#). ArXiv:1909.07547, December.
- 20** Waradpande, Vikram, Daniel Kudenko, and Megha Khosla. 2020. [“Deep Reinforcement Learning with Graph-Based State Representations”](#). ArXiv:2004.13965, April.
- 21** Sharma, Archit, Shixiang Gu, Sergey Levine, Vikash Kumar, and Karol Hausman. 2020. [“Dynamics-Aware Unsupervised Discovery of Skills”](#). ArXiv:1907.01657, February.
- 22** Zheng, Stephan, Alexander Trott, Sunil Srinivasa, Nikhil Naik, Melvin Gruesbeck, David C. Parkes, and Richard Socher. 2020. [“The AI Economist: Improving Equality and Productivity with AI-Driven Tax Policies”](#). ArXiv:2004.13332, April.
- 23** Cox, Tony. 2019. [“Muddling-Through and Deep Learning for Managing Large-Scale Uncertain Risks”](#). *Journal of Benefit-Cost Analysis* 10(2): 226–50.
- 24** Xiao, Kefan, Shiwen Mao, and Jitendra K. Tugnait. 2019. [“TCP-Drinc: Smart Congestion Control Based on Deep Reinforcement Learning”](#). *IEEE Access* 7:(11) 892–904.
- 25** Xiao, Yuchen, Joshua Hoffman, Tian Xia, and Christopher Amato. 2020. [“Learning Multi-Robot Decentralized Macro-Action-Based Policies via a Centralized Q-Net”](#). ArXiv:1909.08776, March.
- 26** Pham, Huy X., Hung M. La, David Feil-Seifer, and Matthew Deans. 2017. [“A Distributed Control Framework for a Team of Unmanned Aerial Vehicles for Dynamic Wildfire Tracking”](#). ArXiv:1704.02630, April.

- 27** Tomic, Teodor, Korbinian Schmid, Philipp Lutz, Andreas Domel, Michael Kassecker, Elmar Mair, Iris Lynne Grix, Felix Ruess, Michael Suppa, and Darius Burschka. 2012. [“Toward a Fully Autonomous UAV: Research Platform for Indoor and Outdoor Urban Search and Rescue”](#). IEEE Robotics Automation Magazine 19(3): 46–56.
- 28** Cui, Jingjing, Yuanwei Liu, and Arumugam Nallanathan. 2018. [“Multi-Agent Reinforcement Learning Based Resource Allocation for UAV Networks”](#). ArXiv:1810.10408, October.
- 29** Lowe, Ryan, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. 2020. [“Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments”](#). ArXiv:1706.02275, March.
- 30** Matignon, Laetitia, Guillaume J. Laurent, and Nadine Le Fort-Piat. 2007. [“Hysteretic Q-Learning: An Algorithm for Decentralized Reinforcement Learning in Cooperative Multi-Agent Teams”](#). *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 64–69.
- 31** Omidshafiei, Shayegan, Jason Pazis, Christopher Amato, Jonathan P. How, and John Vian. 2017. [“Deep Decentralized Multi-Task Multi-Agent Reinforcement Learning under Partial Observability”](#). ArXiv:1703.06182, July.
- 32** Ross, Stephane, Geoffrey J. Gordon, and J. Andrew Bagnell. 2011. [“A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning”](#). ArXiv:1011.0686, March.
- 33** Hester, Todd, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, et al. 2017. [“Deep Q-Learning from Demonstrations”](#). ArXiv:1704.03732, November.
- 34** Ho, Jonathan, and Stefano Ermon. 2016. [“Generative Adversarial Imitation Learning”](#). ArXiv:1606.03476, June.
- 35** Reddy, Siddharth, Anca D. Dragan, and Sergey Levine. 2019. [“SQIL: Imitation Learning via Reinforcement Learning with Sparse Rewards”](#). ArXiv:1905.11108, September.
- 36** Ng, Andrew Y., and Stuart J. Russell. 2000. “Algorithms for Inverse Reinforcement Learning.” *Proceedings of the Seventeenth International Conference on Machine Learning*, 663–670. ICML 2000. San Francisco, CA: Morgan Kaufmann Publishers Inc.

- 37** Tucker, Aaron, Adam Gleave, and Stuart Russell. 2018. [“Inverse Reinforcement Learning for Video Games”](#). ArXiv:1810.10593, October.
- 38** Cai, Xin-Qiang, Yao-Xiang Ding, Yuan Jiang, and Zhi-Hua Zhou. 2020. [“Imitation Learning from Pixel-Level Demonstrations by HashReward”](#). ArXiv:1909.03773, June.
- 39** Shiarlis, Kyriacos, Joao Messias, and Shimon Whiteson. 2016. “Inverse Reinforcement Learning from Failure.” *Proceedings of the 2016 International Conference on Autonomous Agents & Multiagent Systems*, 1060–1068. AAMAS 2016. Singapore: International Foundation for Autonomous Agents and Multiagent Systems.
- 40** Sukhbaatar, Sainbayar, Zeming Lin, Ilya Kostrikov, Gabriel Synnaeve, Arthur Szlam, and Rob Fergus. 2018. [“Intrinsic Motivation and Automatic Curricula via Asymmetric Self-Play”](#). ArXiv:1703.05407, April.
- 41** Laux, Melvin, Oleg Arenz, Jan Peters, and Joni Pajarinen. 2020. [“Deep Adversarial Reinforcement Learning for Object Disentangling”](#). ArXiv:2003.03779, March.
- 42** Jabri, Allan, Kyle Hsu, Ben Eysenbach, Abhishek Gupta, Sergey Levine, and Chelsea Finn. 2019. [“Unsupervised Curricula for Visual Meta-Reinforcement Learning”](#). ArXiv:1912.04226, December.
- 43** Johannink, Tobias, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. 2018. [“Residual Reinforcement Learning for Robot Control”](#). ArXiv:1812.03201, December.
- 44** Alet, Ferran, Martin F. Schneider, Tomas Lozano-Perez, and Leslie Pack Kaelbling. 2020. [“Meta-Learning Curiosity Algorithms”](#). ArXiv:2003.05325, March.
- 45** Wang, Rui, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. 2019. [“Paired Open-Ended Trailblazer \(POET\): Endlessly Generating Increasingly Complex and Diverse Learning Environments and Their Solutions”](#). ArXiv:1901.01753, February.
- 46** Paine, Tom Le, Sergio Gómez Colmenarejo, Ziyu Wang, Scott Reed, Yusuf Aytar, Tobias Pfaff, Matt W. Hoffman, et al. 2018. [“One-Shot High-Fidelity Imitation: Training Large-Scale Deep Nets with RL”](#). ArXiv:1810.05017, October.
- 47** Pham, Huy X., Hung M. La, David Feil-Seifer, and Matthew Deans. 2017. [“A Distributed Control Framework for a Team of Unmanned Aerial Vehicles for Dynamic Wildfire Tracking”](#). ArXiv:1704.02630, April.

- 48** OroojlooyJadid, Afshin, and Davood Hajinezhad. 2020. [“A Review of Cooperative Multi-Agent Deep Reinforcement Learning”](#). ArXiv:1908.03963, June.
- 49** Zhang, Kaiqing, Zhuoran Yang, and Tamer Başar. 2019. [“Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms”](#). ArXiv:1911.10635, November.
- 50** Oliehoek, Frans A., and Christopher Amato. 2016. *A Concise Introduction to Decentralized POMDPs*. 1st ed. Springer Publishing Company.
- 51** Arora, Saurabh, and Prashant Doshi. 2019. [“A Survey of Inverse Reinforcement Learning: Challenges, Methods and Progress”](#). ArXiv:1806.06877, August.
- 52** Narvekar, Sanmit, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. 2020. [“Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey”](#). ArXiv:2003.04960, March.
- 53** Portelas, Rémy, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. 2020. [“Automatic Curriculum Learning For Deep RL: A Short Survey”](#). ArXiv:2003.04664, May.
- 54** Peng, Huimin. 2020. [“A Comprehensive Overview and Survey of Recent Advances in Meta-Learning”](#). ArXiv:2004.11149, June.
- 55** Lazaric, Alessandro. 2012. [“Transfer in Reinforcement Learning: A Framework and a Survey”](#). *Reinforcement Learning: State-of-the-Art*, edited by Marco Wiering and Martijn van Otterlo, 143–73. Adaptation, Learning, and Optimization. Berlin, Heidelberg: Springer.
- 56** Zhuang, Fuzhen, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. 2020. [“A Comprehensive Survey on Transfer Learning”](#). ArXiv:1911.02685, June.